ARTE DE AMAR

DOCUMENTAÇÃO DO SISTEMA

Documentação *completa*

Requisitos, arquitetura, banco de dados, API, testes e manual do painel do site da OSC Arte de Amar.

Site e painel da OSC Arte de Amar
Palmas, Tocantins · Projeto Integrador 1 · Sistemas de Informação · UNITINS

Versão 1.1.1
05 de novembro de 2026

Sumário

01 Requisitos

Atores, restrições, requisitos funcionais e não funcionais.

02 Arquitetura

Camadas, fluxos, edição simultânea e decisões técnicas.

03 Modelo do banco

Diagrama entidade-relacionamento, tabelas, regras e migrations.

04 Referência da API

Rotas, formatos de pedido e resposta, erros e sessão.

05 Plano e relatório de testes

Estratégia em caixa branca, cinza e preta, rastreabilidade e achados.

06 Manual do painel

Como a diretoria usa o painel, em linguagem simples.

CAPÍTULO 01

Requisitos

Atores, restrições, requisitos funcionais e não funcionais.

Site institucional e painel de conteúdo da OSC Arte de Amar (Palmas, TO), levantados com a diretoria e o grupo do Projeto Integrador 1.

Cada requisito tem um identificador. Os testes citam esse identificador no nome (`RF05 · …`), e a matriz de rastreabilidade liga um ao outro.

Atores

ator	quem é	o que faz
Visitante	qualquer pessoa na internet	lê o site, copia o PIX, se inscreve como voluntária
Diretoria	pessoas com login no painel	mantém o conteúdo, acompanha voluntários, gerencia acessos
Mantenedor	quem cuida do código (hoje, a equipe do PI, pelo GitHub)	publica, aplica migrations, recupera o acesso em último caso

Restrições

id	restrição
RE01	Custo zero de operação: só o domínio pode ser pago.
RE02	Hospedagem no Cloudflare (Pages, Functions e D1 no plano gratuito).
RE03	Design autoral aprovado, sem template; o painel não pode quebrar o design.
RE04	A diretoria não é técnica: tudo pelo painel, em português, sem terminal.
RE05	A conta da Cloudflare (hospedagem, banco e domínio) fica em nome da ONG; o código fica no GitHub, com a equipe.

Requisitos funcionais

Site público

id	requisito
RF01	O site mostra projetos, linha do tempo, diretoria e números de impacto vindos do banco.
RF02	Itens marcados como ocultos no painel não aparecem no site; seção sem nenhum item visível sai da página.
RF03	Se a API falhar ou demorar mais de 2,5 s, o site abre com o conteúdo de reserva do código.
RF04	O visitante se inscreve como voluntário (nome, WhatsApp, como quer ajudar); o contato fica guardado.
RF05	Se a API estiver fora do ar, a inscrição abre o WhatsApp da ONG com a mensagem pronta.
RF06	Textos e fotos fixos de cada seção (abertura, quem somos, história, atuação, como ajudar, rodapé) vêm do banco quando a diretoria os trocou.
RF07	WhatsApp e chave PIX exibidos no site são os configurados no painel.
RF21	O site mostra o QR Code do PIX e o "Pix copia e cola", gerados da chave configurada (BR Code estático, sem valor fixo).

Painel

id	requisito
RF10	Só entra no painel quem tem login; a sessão expira em 8 horas.
RF11	A diretoria cria, edita, apaga, oculta e reordena projetos, marcos, pessoas da diretoria e números.
RF12	A diretoria envia fotos, corrige a descrição, apaga as que não estão em uso e baixa uma ou todas (.zip).
RF13	Fotos enviadas são reduzidas (até 1600px) e convertidas para WebP antes de subir.
RF14	A diretoria edita os textos e fotos fixos de cada seção e pode voltar qualquer campo ao original.
RF15	A diretoria vê os voluntários, marca como chamado, arquiva, apaga e chama no WhatsApp.
RF16	A diretoria troca WhatsApp, chave PIX (com prévia do QR Code antes de salvar) e a própria senha.
RF17	Qualquer pessoa da diretoria dá e tira acesso de outra e gera senha temporária para quem esqueceu.
RF18	Quem entra com senha temporária é obrigado a criar uma senha própria antes de usar o painel.
RF19	Toda alteração e todo login ficam registrados (quem, o quê, quando) e são consultáveis por dia e pessoa.
RF20	O painel mostra o espaço usado do banco e avisa a partir de 70%.

Requisitos não funcionais

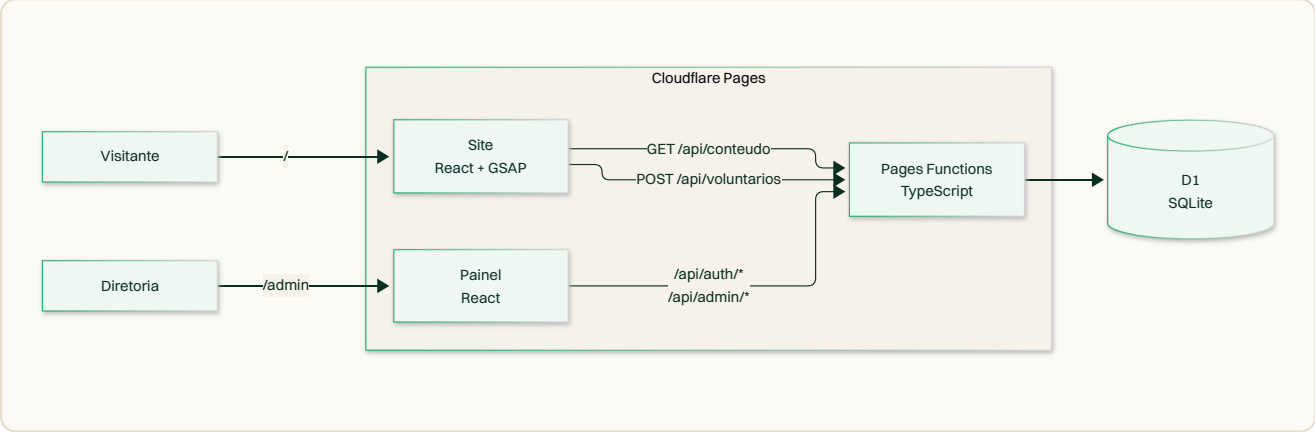
id	categoria	requisito
RNF01	Segurança	Senhas guardadas só como hash PBKDF2-SHA256 com salt por usuário.
RNF02	Segurança	Sessão em cookie `HttpOnly`, `SameSite=Strict` (e `Secure` em HTTPS), assinada (JWT HS256).
RNF03	Segurança	Pedido que altera dados vindo de outra origem é recusado (CSRF).
RNF04	Segurança	Login trava 15 minutos depois de 5 senhas erradas; a mensagem não revela se o e-mail existe.
RNF05	Segurança	Nenhum nome de tabela ou coluna vem do pedido; valores sempre por bind (sem SQL injection).
RNF06	Segurança	Upload aceita só JPG, PNG e WebP conferidos pelos bytes, até 1,5 MB.
RNF07	Segurança	Formulário público com campo-isca anti-robô e até 5 inscrições por hora por IP.
RNF08	Integridade	Duas pessoas editando o mesmo item: quem salva por cima recebe aviso, nada é sobrescrito em silêncio.
RNF09	Integridade	Mudanças de vários registros (reordenar, contatos) são gravadas juntas ou nenhuma.
RNF10	Integridade	Foto em uso não pode ser apagada; dados inválidos são recusados com mensagem em português.
RNF11	Contrato	As respostas da API seguem o formato que o site e o painel esperam (schemas em `tests/contratos.ts`).
RNF12	Usabilidade	Painel em português, utilizável no celular (390px) e no computador.
RNF13	Acessibilidade	Site e painel sem violações graves de acessibilidade (axe, WCAG 2 A/AA).
RNF14	Capacidade	Cabe no plano gratuito do D1 (500 MB): fotos reduzidas, registros temporários limpos sozinhos, auditoria guardada por um ano.
RNF15	Disponibilidade	O site público nunca fica em branco por causa do backend (ver RF03).
RNF16	Manutenção	Esquema do banco versionado em migrations; cada parte testada automaticamente no CI.

CAPÍTULO 02

Arquitetura

Camadas, fluxos, edição simultânea e decisões técnicas.

Um monorepo, um deploy, custo zero (RE01). O Cloudflare Pages publica o site, o painel e a API juntos; o banco é um D1.

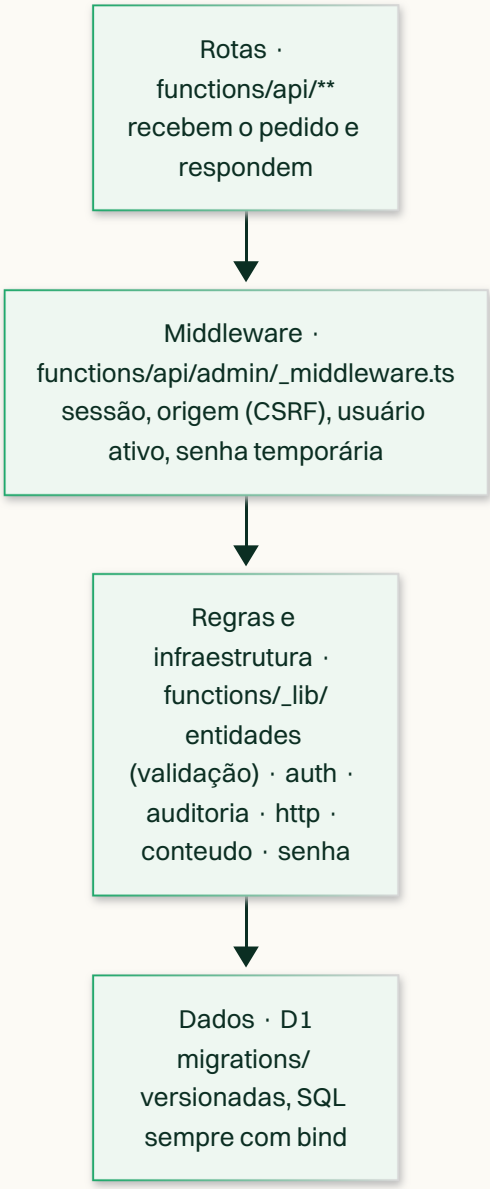


Hospedagem e quem cuida de cada parte

parte	onde fica	quem cuida
site, painel e API	Cloudflare Pages, na conta da ONG	publicação automática a cada push na `main`
banco de dados	Cloudflare D1, na mesma conta	a diretoria, pelo painel
domínio próprio	Cloudflare, na mesma conta	a ONG paga; a equipe configura
código	GitHub (`Gustavo16378/arte-de-amar`)	a equipe

A conta da Cloudflare fica em nome da ONG para o site não depender de ninguém da equipe para continuar no ar (RE05). A equipe só precisa de acesso ao GitHub para publicar mudanças de código.

Camadas

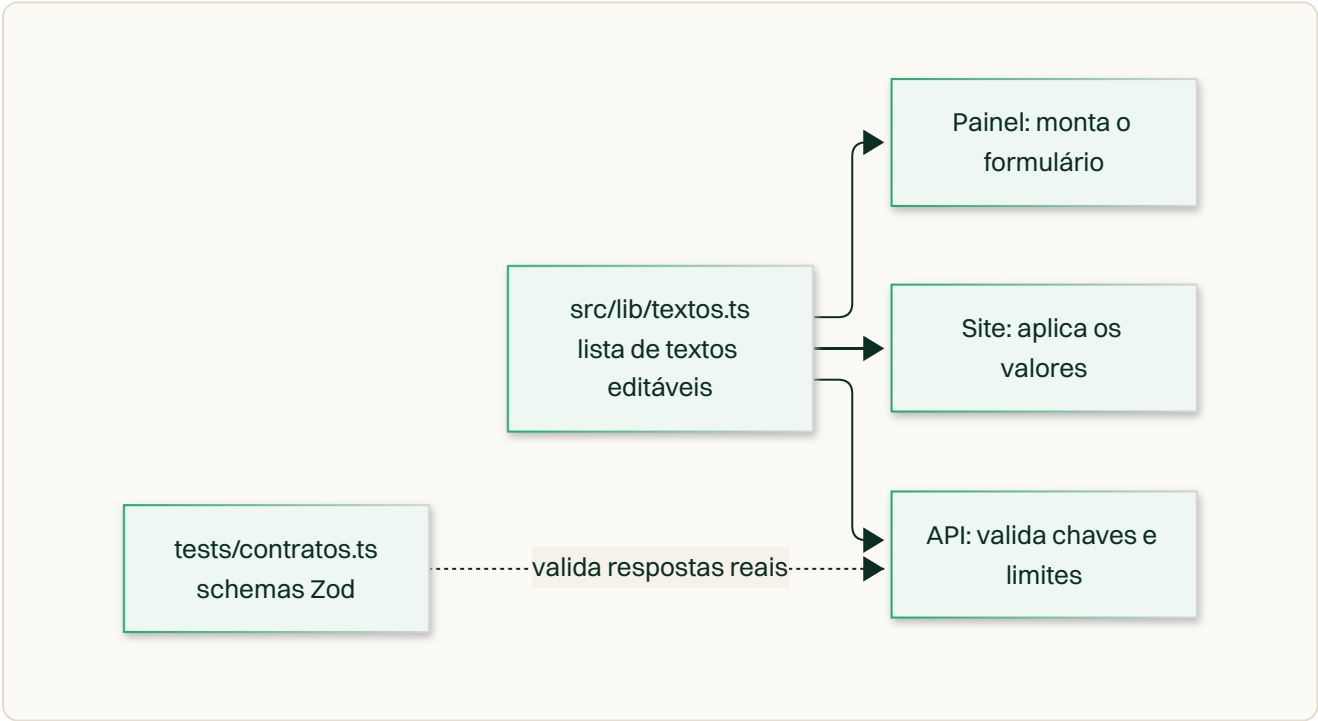


camada	onde	responsabilidade
Rotas	`functions/api/`	Uma rota por arquivo (roteamento do Pages). Leem o pedido, chamam as regras, gravam e respondem.
Middleware	`functions/api/admin/_middleware.ts`	Porteiro de tudo em `/api/admin`: token válido, pessoa ainda cadastrada, origem igual à do site, troca de senha pendente.
Regras	`functions/_lib/entidades.ts`	Campos de cada cadastro, limites, tipos e regras entre campos. Lista fechada: nenhum nome de tabela ou coluna vem do pedido.

camada	onde	responsabilidade
Infraestrutura	`functions/_lib/auth.ts`, `http.ts`, `auditoria.ts`	Hash PBKDF2 e JWT com a Web Crypto do runtime, respostas e erros padronizados, registro de auditoria.
Dados	`migrations/` e D1	Esquema com CHECKs e chaves estrangeiras, para o banco recusar dado ruim mesmo se a API errar.

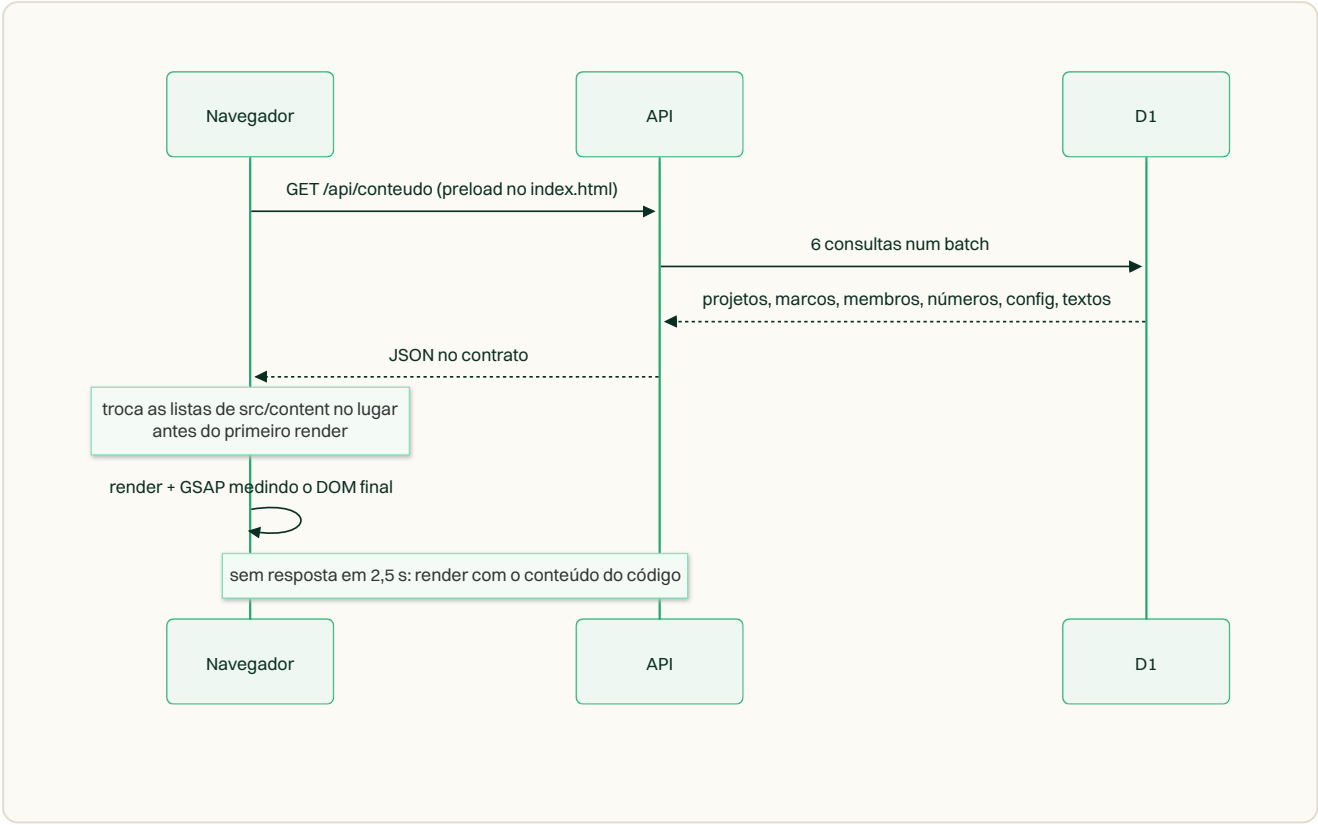
As rotas falam SQL direto com o D1, sempre por bind. Para o tamanho do projeto, uma camada de repositório separada seria mais cerimônia do que ganho; as regras que importam (validação, quem pode o quê, auditoria) já estão fora das rotas.

Uma fonte só para o que é compartilhado



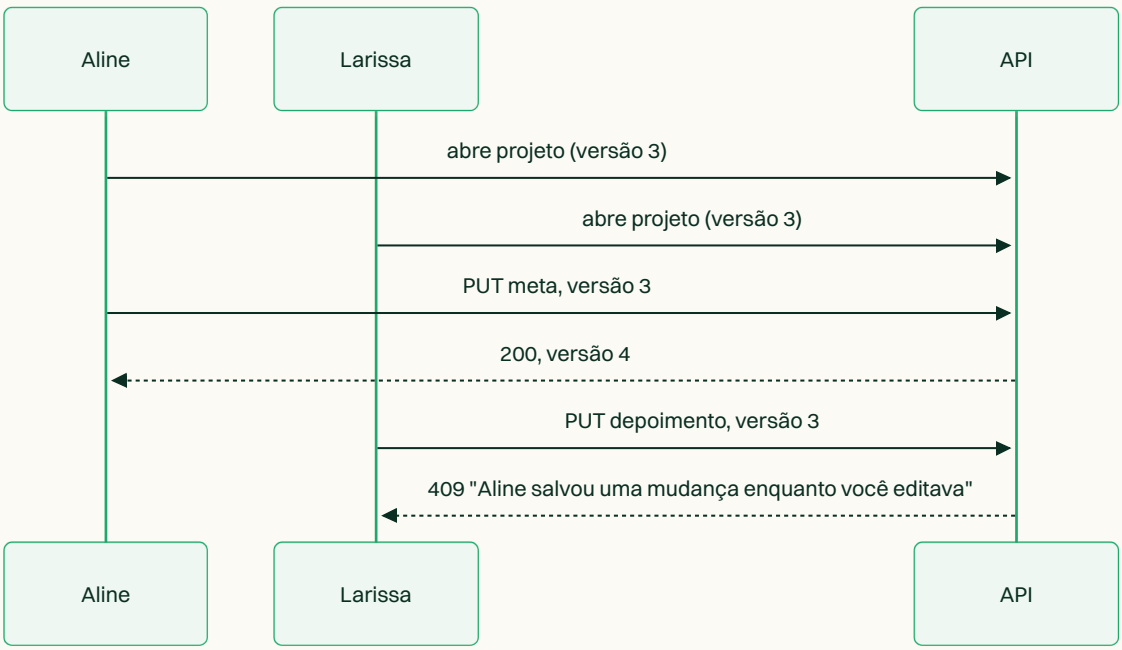
A lista de textos editáveis é importada pelo site, pelo painel e pela API. Os contratos de teste conferem que a API responde no formato que o front espera. Com front e back em repositórios separados, isso viraria cópia mantida à mão.

Fluxo de leitura do site



O conteúdo é aplicado antes do primeiro render de propósito: as seções medem o DOM para montar as animações (a linha do tempo pinada, o mural da diretoria), e conteúdo chegando depois deixaria essas medidas erradas.

Edição simultânea



`UPDATE … SET versao = versao + 1 WHERE id = ? AND versao = ?` : se duas gravações chegam no mesmo instante, o banco deixa passar só uma.

Decisões

decisão	alternativa	por quê
Pages Functions + D1	Spring Boot no Render, Strapi no Railway	Custo zero e sem servidor dormindo; o deploy do back vem junto com o do site.
Fotos no próprio D1	Cloudflare R2	O R2 pede cartão mesmo no plano gratuito. As fotos chegam reduzidas (~0,3 MB).
JWT em cookie HttpOnly	token no localStorage	O JavaScript da página nunca vê o token; SameSite=Strict barra outro site.
Senha temporária pelo painel	recuperação por e-mail	Envio de e-mail exigiria serviço pago ou mais uma conta para manter.
Monorepo	repositórios de front e back	Um deploy, uma fonte para o que é compartilhado, um CI testando tudo junto.

CAPÍTULO 03

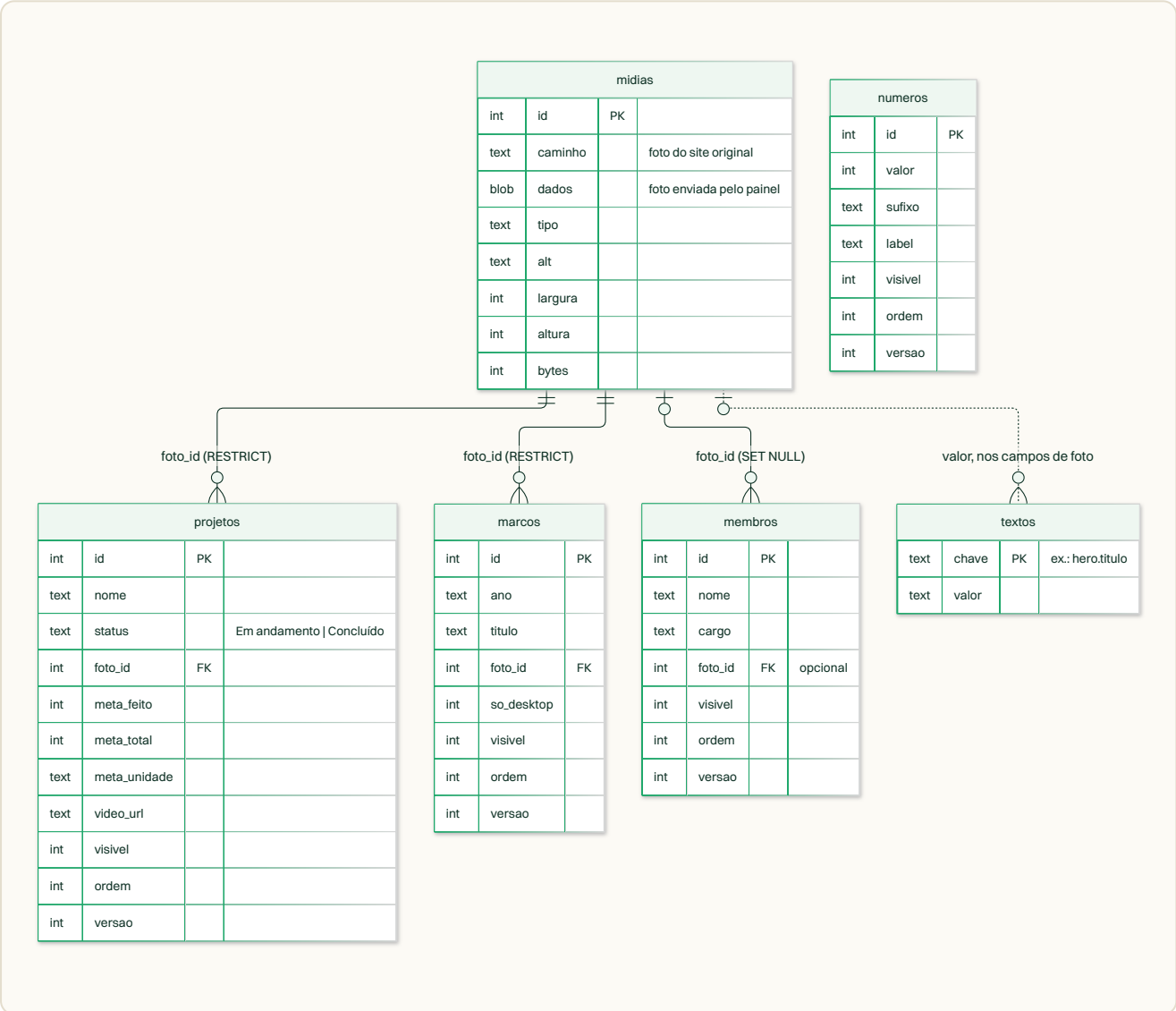
Modelo do banco

Diagrama entidade-relacionamento, tabelas, regras e migrations.

Cloudflare D1 (SQLite). O esquema nasce das migrations em `migrations/`, aplicadas em ordem; nenhuma tabela é criada à mão.

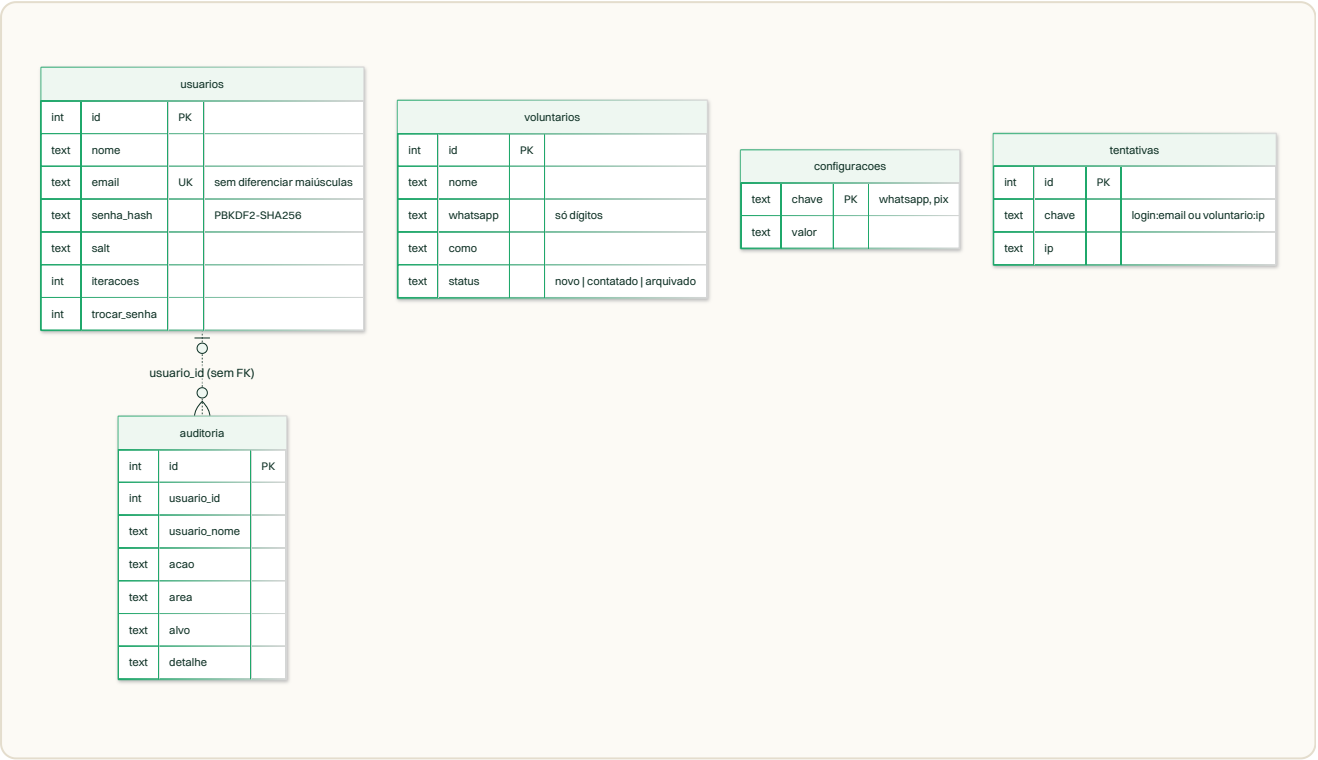
Diagrama entidade-relacionamento

Conteúdo do site



Painel e operação

`auditoria` guarda o id e o nome de quem fez, sem chave estrangeira: o registro continua legível depois que a pessoa sai. `voluntarios`, `configuracoes` e `tentativas` não se ligam a outras tabelas.



Tabelas

tabela	guarda	observações
`midias`	fotos	Ou `caminho` (arquivo do site original) ou `dados` (imagem enviada); um dos dois é obrigatório.
`projetos`, `marcos`, `membros`, `numeros`	os cadastros do site	`ordem` define a posição no site, `visivel` esconde sem apagar, `versao` protege a edição simultânea.
`textos`	textos e fotos fixos trocados no painel	Só existe linha para o que foi trocado; o resto fica com o original do código.
`configuracoes`	WhatsApp e chave PIX	Chave e valor.
`voluntarios`	inscrições do site	Telefone só com dígitos.
`usuarios`	quem entra no painel	Nunca guarda senha, só hash, salt e iterações.
`auditoria`	quem fez o quê e quando	Guarda o nome por escrito: o registro continua legível depois que a pessoa sai. Apagada depois de um ano.
`tentativas`	freio de login e de inscrição	Apagada depois de um dia.

Regras que o próprio banco garante

Mesmo com um bug na API, o banco recusa:

- status, `visivel`, `so_desktop`, `trocar_senha` e ação da auditoria fora dos valores previstos (CHECK);
- meta negativa ou total zero; foto com dimensão zero; foto sem arquivo e sem bytes;
- e-mail repetido, inclusive com maiúsculas diferentes (UNIQUE NOCASE);
- projeto ou marco apontando para foto inexistente, e apagar foto em uso (FOREIGN KEY RESTRICT).

Apagar o retrato de uma pessoa da diretoria deixa a pessoa sem foto (SET NULL), e o site mostra a moldura com o cargo.

Tudo isso é testado em `tests/db/schema.test.ts`.

Migrations

arquivo	o que faz
`0001_schema.sql`	esquema inicial e índices
`0002_seed.sql`	conteúdo aprovado do site, gerado de `src/content` por `scripts/gerar-seed.ts`
`0003_diretoria.sql`	diretoria real com retratos; cargos sem nome ocultos
`0004_textos.sql`	tabela dos textos editáveis
`0005_usuarios_auditoria.sql`	senha temporária, auditoria e coluna de versão

Nova mudança de esquema é sempre uma migration nova, com o próximo número. Migration já aplicada em produção não se edita.

```
npm run db:local    # aplica no banco de desenvolvimento
npm run db:remoto   # aplica no banco de produção
```

Capacidade

O plano gratuito aceita 500 MB por banco. Quase todo o peso são as fotos enviadas (cerca de 0,3 MB cada); um contato de voluntário tem menos de 1 KB. O painel mostra o espaço usado em Configurações e avisa a partir de 70%.

CAPÍTULO 04

Referência da API

Rotas, formatos de pedido e resposta, erros e sessão.

Base: `https://arte-de-amar-site.pages.dev/api`. Tudo em JSON e UTF-8, exceto o upload de foto (multipart) e a entrega de foto (a imagem).

Os formatos abaixo são os mesmos validados pelos contratos de `tests/contratos.ts`.

Convenções

Erro. Todo erro sai no mesmo formato, com a mensagem em português, pronta para mostrar na tela:

```
{ "erro": "Preencha o campo \"Nome\"." }
```

status	quando
400	dado inválido (a mensagem diz qual campo)
401	sem sessão, sessão expirada ou login errado
403	pedido de outra origem (CSRF) ou senha temporária ainda não trocada
404	rota, recurso ou registro inexistente
405	método não suportado na rota
409	conflito: outra pessoa salvou antes, foto em uso, e-mail repetido
413	corpo ou arquivo grande demais
415	`Content-Type` errado ou formato de imagem não aceito
429	muitas tentativas (login ou inscrição)
500	erro do servidor, sem detalhe interno na resposta

Sessão. O login devolve o cookie `ada_sessao` (`HttpOnly`, `SameSite=Strict`, `Secure` em HTTPS, `Path=/api`, 8 horas). O navegador manda sozinho; nenhuma rota recebe token no corpo ou no cabeçalho.

CSRF. Em `/api/admin/*`, todo método que altera dados com cabeçalho `Origin` diferente do site é recusado com 403.

Datas. Sempre UTC, no formato do SQLite: `2026-11-03 16:53:22`.

Público

`GET /conteudo`

Tudo que o site mostra e a diretoria edita, numa ida ao banco. Só itens visíveis, na ordem do painel. Sem cache.


```
{
  "projetos": [
    {
      "nome": "Volta às Aulas",
      "objetivo": "Mochila completa e material escolar entregue em mãos…",
      "objetivoCurto": "Mochila completa e material entregue em mãos…",
      "publico": "Estudantes da rede pública",
      "status": "Em andamento",
      "foto": { "src": "/assets/fotos/hero-menino-mochila.jpg", "alt": "Menino sorrindo…",
"largura": 1600, "altura": 2400, "posicao": "50% 30%" },
      "meta": [86, 120, "mochilas"],
      "depoimento": "“Ele escolheu a mochila dos Minions…”",
      "autor": "Avó atendida, 2026",
      "videoUrl": null
    }
  ],
  "marcos": [{ "ano": "2015", "titulo": "A fundação", "texto": "…", "textoDesktop": "…",
"foto": { "…": "…" }, "soDesktop": false }],
  "membros": [{ "nome": "Aline Milani", "cargo": "Presidente", "foto": { "…": "…" } }],
  "numeros": [{ "valor": 11, "sufixo": "", "label": "anos de atuação" }],
  "config": { "whatsapp": "5563999999999", "pix": "40.019.106/0001-51" },
  "textos": { "hero.subtitulo": "Texto trocado no painel", "footer.foto": { "src":
"/api/midias/14", "…": "…" } }
}
```


`meta` é `[feito, total, unidade]` ou `null`. Em `textos` só aparecem os campos que a diretoria trocou; campo de foto já vem como objeto.

`POST /voluntarios`

Formulário "Ser voluntário".


```
{ "nome": "Maria Souza", "whatsapp": "(63) 98877-6655", "como": "Natal Solidário", "site":
"" }
```


`201 { "ok": true }`. O telefone é guardado só com dígitos. `site` é o campo-isca: se vier preenchido, a resposta é a mesma e nada é gravado. Limite de 5 inscrições por hora por IP (429).

`GET /midias/:id`

A foto enviada pelo painel, com `Cache-Control: immutable` de um ano e `X-Content-Type-Options: nosniff`. Foto do site original responde 302 para o arquivo em `/assets/fotos`. `?baixar=1` entrega como download, com nome tirado da descrição.

Sessão

`POST /auth/login`

```
{ "email": "aline@ong.org", "senha": "minha-senha" }
```

`200 { "usuario": { "id": 2, "nome": "Aline Milani", "email": "aline@ong.org", "trocarSenha": false } }` e o cookie. Senha errada e e-mail inexistente dão a mesma resposta (401). Depois de 5 erros no mesmo e-mail em 15 minutos, 429.

`POST /auth/logout`

Apaga o cookie. `200 { "ok": true }`.

`GET /auth/eu`

Quem está logado, no mesmo formato do login, ou 401.

Painel (`/admin`, exige sessão)

Com senha temporária pendente, tudo devolve 403, exceto `PUT /admin/senha`.

Cadastros: `projetos`, `marcos`, `membros`, `numeros`

método	rota	corpo	resposta
GET	`/admin/:entidade`		`{ itens: [...] }`, inclusive os ocultos, com `foto_url`
POST	`/admin/:entidade`	os campos	`201 { item }`, no fim da lista, versão 0
PUT	`/admin/:entidade/:id`	só os campos alterados + `versao`	`{ item }` com a versão nova
DELETE	`/admin/:entidade/:id`		`{ ok: true }`

método	rota	corpo	resposta
PUT	`/admin/:entidade/ordem`	`{ "ids": [3, 1, 2] }`	`{ ok: true }`

Versão (edição simultânea). O `PUT` com `versao` só grava se ela for a atual; senão responde 409 com quem salvou antes. Sem `versao` (o interruptor "No site"), vale o último.

```
PUT /api/admin/projetos/2
{ "meta_feito": 90, "versao": 4 }

409 { "erro": "Larissa Lima salvou uma mudança nesse projeto enquanto você editava. Feche, abra de novo…" }
```

Campos de cada cadastro (limites validados na API e no banco):

entidade	campos
projetos	`nome` * (80), `objetivo` * (400), `objetivo_curto` * (200), `publico` * (80), `status` * (`Em andamento` | `Concluído`), `foto_id` *, `foto_posicao` , `meta_feito` , `meta_total` , `meta_unidade` (os três juntos ou nenhum), `depoimento` (300), `autor` (80), `video_url` (https), `visivel`
marcos	`ano` * (10), `titulo` * (60), `texto` * (200), `texto_desktop` * (300), `foto_id` *, `foto_posicao` , `so_desktop` , `visivel`
membros	`nome` * (60), `cargo` * (60), `foto_id` , `visivel`
numeros	`valor` * (inteiro ≥ 0), `sufixo` (3), `label` * (40), `visivel`

* obrigatório. Entre parênteses, o máximo de caracteres.

Fotos

método	rota	corpo	resposta
GET	`/admin/midias`		`{ itens: [{ id, url, alt, largura, altura, bytes, estatica, usos, criado_em }] }`
POST	`/admin/midias`	multipart: `arquivo` , `alt` , `largura` , `altura`	`201 { item }`
PUT	`/admin/midias/:id`	`{ "alt": "…" }`	`{ ok: true }`
DELETE	`/admin/midias/:id`		`{ ok: true }` , ou 409 se em uso ou do site original

O tipo é conferido pelos primeiros bytes (JPEG, PNG, WebP). SVG e qualquer outra coisa: 415. Acima de 1,5 MB: 413. O painel já manda a foto reduzida para 1600px e em WebP.

Textos do site

método	rota	corpo	resposta
GET	`/admin/textos`		`{ valores: { chave: valor }, fotos: { chave: url } }`
PUT	`/admin/textos`	`{ "valores": { "hero.titulo": "Linha 1\nLinha |2|", "hero.foto": "14", "footer.frase": null } }`	`{ ok: true }`

As chaves válidas e os limites estão em `src/lib/textos.ts`. `null` volta o campo ao original. Um campo inválido barra o pacote inteiro.

Voluntários

método	rota	corpo	resposta
GET	`/admin/voluntarios`		`{ itens }`, novos primeiro
PUT	`/admin/voluntarios/:id`	`{ "status": "novo" | "contatado" | "arquivado" }`	`{ ok: true }`
DELETE	`/admin/voluntarios/:id`		`{ ok: true }`
DELETE	`/admin/voluntarios`		`{ apagados: n }`, só os arquivados

Usuários

método	rota	corpo	resposta
GET	`/admin/usuarios`		`{ itens: [{ id, nome, email, trocar_senha, criado_em, ultimo_acesso }] }`
POST	`/admin/usuarios`	`{ "nome": "…", "email": "…" }`	`201 { item, senhaTemporaria }`
PUT	`/admin/usuarios/:id`		`{ senhaTemporaria }` (esqueceu a senha)
DELETE	`/admin/usuarios/:id`		`{ ok: true }`; não vale para si mesmo nem para o último
PUT	`/admin/senha`	`{ "atual": "…", "nova": "…" }`	`{ ok: true }`; com senha temporária, `atual` não é pedida

A senha temporária tem o formato `kq7m-x3ra-p9ve` e aparece só nessa resposta. Tirar o acesso derruba a sessão da pessoa no próximo pedido.

Outros

método	rota	resposta
GET, PUT	`/admin/configuracoes`	`{ config: { whatsapp, pix } }`; o PUT completa o 55 do WhatsApp
GET	`/admin/auditoria?de=2026-11-03 03:00:00&ate=2026-11-04 03:00:00`	`{ itens: [{ id, usuario_id, usuario_nome, acao, area, alvo, detalhe, criado_em }] }`, até 2000
GET	`/admin/espaco`	`{ usado, limite, porcentagem, bytesFotos, fotosEnviadas, voluntarios }`

CAPÍTULO 05

Plano e relatório de testes

Estratégia em caixa branca, cinza e preta, rastreabilidade e achados.

Como o sistema é verificado, o que cada suíte cobre e como cada requisito de requisitos.md é testado.

Resumo

suíte	técnica	o que roda	testes
Unitários	caixa branca	bibliotecas da API e do site, isoladas	121
Banco	caixa branca	migrations aplicadas do zero num SQLite em memória	34
API	caixa preta e caixa cinza	a API real (wrangler, runtime da Cloudflare) com um D1 isolado	155
E2E	caixa preta	site e painel num navegador, no computador (1440px) e no celular (390px)	48
Acessibilidade	caixa preta	axe-core, regras WCAG 2 A e AA, dentro do E2E	incluída no E2E
Total			**358**

Cobertura de linhas dos unitários: 89% nas bibliotecas (`functions/_lib` , `src/lib` , zip do painel). O que fica de fora do unitário (`auditoria.ts` e `conteudo.ts`) depende do banco e é coberto pela suíte de API.

As três técnicas

Caixa branca conhece o código e testa os caminhos de dentro: cada ramo da validação, o hash de senha, a assinatura do token, a troca do conteúdo de reserva, o formato binário do zip. Fica em `tests/unit/` e `tests/db/` .

Caixa preta só conhece a interface: manda um pedido HTTP ou clica na tela e confere a resposta, sem saber como foi feito. É a maior parte de `tests/api/` e todo o `tests/e2e/` . Os contratos em `tests/contratos.ts` (schemas Zod) definem o que a API promete ao site e ao painel.

Caixa cinza faz o pedido pela interface, como a preta, mas depois abre o banco do servidor de teste (só leitura) e confere o que foi gravado: que a senha virou hash, que a auditoria registrou quem fez, que a versão subiu, que o dado inválido não chegou ao banco. Os testes marcados "caixa cinza" em `tests/api/` fazem isso pela função `banco()` de `tests/api/cliente.ts` .

Ambiente

Nenhum teste toca o banco de desenvolvimento nem o de produção. `tests/servidor-teste.mjs` cria, a cada rodada, uma pasta temporária, aplica todas as migrations, cria um usuário de teste e sobe o `wrangler pages dev` apontado para ela. Os testes de API e os E2E usam esse servidor; ao terminar, a pasta é apagada.

Os arquivos da suíte de API contam uma história em ordem (01-contrato até 09-seguranca) sobre um banco só, por isso rodam em sequência, pelo nome.

Como rodar

```
npm run build      # a API e os E2E servem o dist/
npm test           # unitários + banco + API
npm run test:e2e   # E2E e acessibilidade (baixe o navegador uma vez: npx playwright install chromium)
npm run test:tudo  # tudo, na ordem
npm run test:cobertura
```

No CI (`.github/workflows/testes.yml`) a pirâmide inteira roda a cada push e pull request na `main`. Se o E2E falhar, o relatório do Playwright, com trace e screenshot do erro, fica salvo como artefato.

O que cada suíte testa

Unitários (`tests/unit/`)

arquivo	foco
`auth.test.ts`	hash PBKDF2 (salt, compatibilidade com o script de admin), JWT (adulterado, outro segredo, expirado em 8 h, malformado), flags do cookie
`entidades.test.ts`	lista fechada de tabelas, cada regra de validação, a regra da meta, campos alterados para a auditoria
`http.test.ts`	leitura de JSON (tipo, tamanho, formato), formato único de erro, erro interno sem vazar detalhe, ids de rota
`textos.test.ts`	chaves únicas, texto original dentro do limite, aplicação no site sem mexer no catálogo de fotos
`conteudo-remoto.test.ts`	troca do conteúdo no lugar, layout derivado da ordem, reserva com API em erro, fora do ar, lenta (2,5 s) ou fora do contrato

arquivo	foco
`pix.test.ts`	CRC16 com o valor de verificação padrão, o payload do manual do Pix do Banco Central reproduzido caractere por caractere, normalização das chaves (CNPJ, CPF, telefone, e-mail, aleatória), limites de nome e cidade
`utilitarios.test.ts`	zip relido byte a byte e conferido com o CRC-32 do zlib (e pelo Python, quando instalado), nomes de arquivo, senha temporária (formato, sem caracteres ambíguos, sem repetição)

Banco (`tests/db/schema.test.ts`)

Migrations em sequência e aplicáveis do zero, `integrity_check`, seed, e o que o banco garante sozinho mesmo se a API errar: CHECKs de status e valores, e-mail único sem diferenciar maiúsculas, foto em uso protegida por chave estrangeira, retrato apagado vira "sem foto" sem apagar a pessoa, versão inicial e UPDATE condicional, índices usados pela consulta da auditoria.

API (`tests/api/`)

arquivo	foco
`01-contrato`	`/api/conteudo` dentro do contrato, seed, nada interno exposto, formato único de erro
`02-autenticacao`	login, mesma mensagem para e-mail inexistente e senha errada, trava depois de 5 erros, cookie, logout, as 26 rotas do painel exigindo login
`03-cadastros`	criar, editar, ocultar, reordenar e apagar refletindo no site; validação; rotas e ids com injeção de SQL; configurações
`04-fotos`	upload de JPEG, PNG e WebP reais; tipo pelos bytes; SVG, arquivo disfarçado e arquivo grande recusados; download; proteção de foto em uso
`05-voluntarios`	inscrição, normalização do telefone, campo-isca, limite por IP, status, apagar arquivados de uma vez
`06-textos`	editar, foto de seção, voltar ao original, normalização, limites, pacote inválido não grava pela metade
`07-concorrencia`	409 com o nome de quem salvou antes, dez gravações simultâneas com exatamente uma vencendo, itens diferentes ao mesmo tempo
`08-usuarios-auditoria`	senha temporária, troca obrigatória, sessão derrubada ao tirar acesso, auditoria por período, espaço do banco
`09-seguranca`	CSRF, XSS armazenado, corpo grande, tipo errado, prototype pollution, método não suportado, erro sem detalhe interno

E2E (`tests/e2e/`)

arquivo	foco
`site.spec.ts`	site sem erro de console nem rolagem lateral; reserva com API fora e lenta; inscrição chegando ao painel; erro no formulário; WhatsApp quando a API cai; HTML de texto editado não executa; seção vazia some; QR do PIX lido do print da tela (jsQR) igual ao copia e cola; chave trocada no painel troca o QR
`painel.spec.ts`	login, editar meta e ver no site, editar e voltar texto ao original, upload real (foto de 3000px chega com 1600px), duas pessoas no mesmo projeto, dar acesso e trocar senha obrigatória, auditoria, WhatsApp marcando como chamado, todas as abas no celular
`acessibilidade.spec.ts`	axe-core no site, no login e em sete abas do painel

Rastreabilidade

Cada teste cita no nome o requisito que verifica. Contagem de testes (ou grupos parametrizados) por requisito:

requisito	unit	banco	API	E2E
RF01 conteúdo do banco no site	✓	✓	✓	✓
RF02 ocultos e seção vazia	✓		✓	✓
RF03 conteúdo de reserva	✓			✓
RF04 inscrição de voluntário	✓		✓	✓
RF05 WhatsApp quando a API cai	✓			✓
RF06 textos editados no site	✓		✓	✓
RF07 WhatsApp e PIX configuráveis	✓		✓	
RF21 QR Code e Pix copia e cola	✓			✓
RF10 login e sessão			✓	✓
RF11 cadastros	✓		✓	✓
RF12 fotos	✓		✓	✓
RF13 redução e WebP				✓
RF14 textos e voltar ao original	✓		✓	✓
RF15 voluntários no painel			✓	✓
RF16 configurações e senha			✓	
RF17 acessos e senha temporária	✓		✓	✓

requisito	unit	banco	API	E2E
RF18 troca obrigatória			✓	✓
RF19 auditoria	✓		✓	✓
RF20 espaço do banco			✓	✓
RNF01 hash de senha	✓		✓	
RNF02 sessão e cookie	✓		✓	
RNF03 CSRF			✓	
RNF04 trava de login			✓	
RNF05 SQL injection	✓		✓	
RNF06 upload seguro			✓	
RNF07 anti-robô e limite			✓	
RNF08 edição simultânea		✓	✓	✓
RNF09 tudo ou nada		✓	✓	
RNF10 dado inválido recusado	✓	✓	✓	
RNF11 contratos	✓		✓	
RNF12 painel no celular				✓
RNF13 acessibilidade				✓
RNF14 capacidade		✓	✓	
RNF15 site nunca em branco	✓			✓
RNF16 migrations e CI		✓		

O que os testes encontraram

Problemas reais achados ao escrever esta suíte, todos corrigidos:

achado	onde	correção
Foto usada num texto do site (abertura, rodapé) podia ser apagada. O D1 manda números do JavaScript como REAL, e `CAST(16.0 AS TEXT)` vira `'16.0'`, que não bate com `'16'`.	`functions/api/admin/midias`	comparar convertendo o texto para inteiro: `CAST(valor AS INTEGER) = ?`
Editar só um campo da meta de um projeto dava erro de validação, porque a regra "três campos juntos" rodava sobre o pedido parcial.	`functions/_lib/entidades.ts`	a regra entre campos roda só sobre o registro já mesclado

achado	onde	correção
A auditoria do dia cortava em 500 registros.	`functions/api/admin/auditoria.ts`	limite de 2000 por consulta
Abas do painel com `aria-checked` em `role="tab"` (atributo inválido).	`src/admin/Textos.tsx`	só `aria-selected`
Dicas, horários e botões de apagar do painel abaixo do contraste AA.	`src/admin/admin.css`	tons mais escuros só no painel, o design do site intacto
O mural da diretoria, que rola de lado no celular, não era alcançável pelo teclado.	`Diretoria.tsx`	`tabIndex` e rótulo na lista
Com todos os números ocultos, o site voltava a mostrar todos (achado no uso, virou teste de regressão).	`src/lib/conteudo-remoto.ts`	lista vazia é resposta válida e a seção sai da página
O teste do zip dependia do Python e falhava no Windows sem ele.	`tests/unit/utilitarios.test.ts`	leitura do zip em TypeScript conferida com o CRC-32 do zlib; o Python vira conferência extra opcional

Fora do escopo, por enquanto

- Testes de carga: o tráfego esperado de uma ONG local cabe com folga no plano gratuito (5 milhões de leituras por dia).
- Navegadores além do Chromium: o E2E roda em Chromium no computador e no celular. Firefox e WebKit (Safari) entram adicionando projetos no `playwright.config.ts`.
- Mutação de código (mutation testing) e testes visuais por screenshot.

CAPÍTULO 06

Manual do painel

Como a diretoria usa o painel, em linguagem simples.

O painel é onde vocês cuidam do site sem depender de ninguém. Ele fica em https://arte-de-amar-site.pages.dev/admin e funciona no computador e no celular.

Tudo que vocês salvam aparece no site na próxima vez que alguém abrir a página. Se o site já estiver aberto, é só recarregar.

Entrar

1. Abra o endereço do painel.
2. Digite seu e-mail e sua senha e toque em **Entrar**.

Se você recebeu uma **senha temporária** (aquela no formato `kq7m-x3ra-p9ve`), o painel vai pedir para você criar uma senha sua logo depois de entrar. Use pelo menos 10 caracteres.

Por segurança, a sessão dura 8 horas. Depois disso, é só entrar de novo. Errar a senha 5 vezes seguidas trava o login por 15 minutos.

Primeiro acesso: o que conferir

Na primeira vez, vale passar por tudo com o site aberto em outra aba, para ver cada mudança aparecer (é só recarregar o site depois de salvar).

1. Entre com a senha temporária e crie a sua.
2. Em **Textos do site**, leia cada seção e corrija o que estiver diferente do jeito que a ONG fala.
3. Em **Projetos**, confira nomes, metas, fotos e links de vídeo.
4. Em **Linha do tempo**, **Diretoria** e **Números**, confira anos, nomes, cargos, retratos e valores.
5. Em **Configurações**, confira o WhatsApp e a chave PIX e escaneie a prévia do QR Code no app do banco (veja **PIX**, mais abaixo).
6. Em **Usuários**, dê acesso para as outras pessoas da diretoria que vão cuidar do site.

O que não der para resolver pelo painel, como mudar o layout ou criar uma seção nova, é com a equipe (veja **Quem cuida do quê**, no fim).

Esqueci a senha

Peça para outra pessoa da diretoria:

1. Entrar em **Usuários**.
2. Tocar em **Esqueceu a senha** no seu nome e confirmar.
3. Tocar em **Copiar mensagem para o WhatsApp** e mandar para você.

Você entra com a senha temporária e cria uma nova. Se ninguém conseguir entrar, fale com a equipe.

Textos do site

Os textos fixos de cada parte do site: a abertura, quem somos, a história, onde a gente chega, como ajudar e o rodapé.

1. Escolha a seção nas abas de cima.
2. Mude o texto do campo.
3. Toque em **Salvar** no fim da seção.

Algumas dicas:

- A palavra entre barras retas fica em itálico verde, como no design. Por exemplo, `Nossos |projetos|` aparece com "projetos" em destaque.
- No título da abertura, cada linha do campo é uma linha do título.
- Alguns campos dizem "aparece no celular" ou "aparece no computador". O site usa textos mais curtos no celular.
- Errou? **Voltar ao original** devolve o texto do design. É preciso salvar depois.
- **Trocar foto** abre a biblioteca de fotos. Prefira foto em pé para a abertura.

Projetos, linha do tempo, diretoria e números

As quatro abas funcionam do mesmo jeito.

- **Criar:** toque em **+ Novo** no alto, preencha e salve. O item entra no fim da lista.
- **Editar:** toque no nome do item.
- **Mudar a ordem:** use as setas ↑ ↓. A ordem da lista é a ordem do site.
- **Esconder sem apagar:** desligue a chave **No site**. Para mostrar de novo, é só ligar.
- **Apagar:** abra o item, toque em **Apagar** e confirme. Não tem volta.

Nos **projetos**, a meta é opcional. Preenchendo os três campos (já feito, total e unidade), o card ganha a barra de progresso. O link de vídeo aceita YouTube ou Instagram.

Na **diretoria**, a pessoa sem retrato aparece com uma moldura e o cargo.

"Outra pessoa salvou enquanto você editava"

Se duas pessoas abrirem o mesmo item e as duas salvarem, a segunda recebe esse aviso em vez de apagar o que a primeira fez. Feche o formulário, abra de novo para ver a versão nova e refaça a sua mudança.

Fotos

- **Enviar:** toque em **Enviar uma foto nova**, escolha a foto e escreva uma descrição curta do que aparece nela (é o que pessoas cegas ouvem no leitor de tela). O painel diminui a foto sozinho antes de mandar.
- **Baixar:** toque na foto e em **Baixar**.
- **Baixar todas (.zip):** guarda uma cópia de todas as fotos no seu aparelho. Vale fazer de vez em quando, como cópia de segurança.
- **Apagar:** só dá para apagar foto que não está em uso. Se ela estiver num projeto, na linha do tempo ou nos textos do site, troque lá antes.

Voluntários

Aqui chegam as pessoas que preencheram **Ser voluntário** no site.

- **WhatsApp** abre uma conversa com a pessoa, já com uma mensagem de boas vindas. Ela passa a aparecer como "Já chamado".
- **Arquivar** tira da lista principal sem apagar. Os arquivados ficam na aba **Arquivados**.
- **Apagar todos os arquivados** limpa a lista de uma vez.

Usuários

Quem tem acesso ao painel.

- **Dar acesso a alguém:** nome, e-mail e **Criar acesso**. O painel mostra uma senha temporária para você mandar à pessoa.
- **Tirar acesso:** a pessoa sai do painel na hora, em qualquer aparelho.

Ninguém consegue tirar o próprio acesso, e o painel sempre fica com pelo menos uma pessoa.

Auditoria

Mostra tudo que foi feito no painel em cada dia: quem fez, o quê e a que horas. Escolha o dia no calendário e, se quiser, a pessoa. Fica guardado por um ano.

Configurações

- **WhatsApp da ONG:** o número dos botões "Falar no WhatsApp" do site.
- **Chave PIX:** a chave que gera o QR Code e o Pix copia e cola do site (veja **PIX**, logo abaixo).
- **Espaço do banco:** quanto do espaço gratuito já foi usado. Se passar de 70%, aparece um aviso no alto do painel. Para liberar espaço, apague fotos que não usam mais e voluntários arquivados.
- **Trocar senha:** a atual e a nova duas vezes.

PIX

Em "Como ajudar", o site mostra três coisas, todas geradas da chave PIX salva em **Configurações**:

- **QR Code:** quem está no computador escaneia com o app do banco.
- **Copiar Pix copia e cola:** para quem está no celular, que não consegue escanear a própria tela. A pessoa copia o código e cola no app do banco.
- **A chave escrita**, com o botão de copiar.

O PIX não tem valor fixo: quem doa escolhe quanto quer mandar. A chave pode ser CNPJ, CPF, telefone, e-mail ou chave aleatória.

Ao trocar a chave, o painel mostra uma **prévia do QR Code** logo abaixo do campo. Depois de salvar, escaneie a prévia no app do banco e confira se aparece o nome da conta da ONG antes de divulgar. Se aparecer outro nome, a chave está errada.

Quem cuida do quê

- **Conteúdo do site** (textos, fotos, projetos, linha do tempo, diretoria, números, WhatsApp e PIX): a diretoria, pelo painel.
- **Hospedagem:** o site, o painel e o banco de dados ficam numa conta da Cloudflare em nome da ONG, no plano gratuito. O domínio próprio, quando for comprado, fica na mesma conta.
- **Código:** fica no GitHub, onde a equipe guarda o projeto e faz as atualizações. Mudanças de layout, seções novas e correções passam pela equipe.